

Algoritmo e Programação

Lista Endereços e ponteiros

Exercícios 01

Ex01: Tamanhos. Compile e execute o seguinte programa:

```
//Entrada
#include <stdio.h>
int main (void) {
    typedef struct {
        int dia, mes, ano;
    } data;
    printf ("sizeof (data) = %d\n",
           sizeof (data));
    return 0;
}
```

```
//Saída
sizeof (data) = 12
```

Ex02: Compile e execute o seguinte programa. (O cast(long int) é necessário para que &i possa ser impresso no formato %ld. É mais comum imprimir endereços em notação hexadecimal, usando formato %p, que exige o cast (void *).)

```
//Entrada
#include <stdio.h>
int main (void) {
    int i = 1234;
    printf (" i = %d\n", i);
    printf ("&i = %ld\n", (long int) &i)
    printf ("&i = %p\n", (void *) &i);
    return 0;
}
```

```
//Saída
i = 1234
&i = 6422300
&i = 0061FF1C
```

Exercícios 02

Ex01: Compile e execute o seguinte programa. (Veja um dos exercícios acima.)

```
#include <stdio.h>
#include <stdlib.h>
int main (void) {
    int i; int *p;
    i = 1234; p = &i;
    printf ("*p = %d\n", *p);
    printf (" p = %ld\n", (long int) p);
    printf (" p = %p\n", (void *) p);
    printf ("&p = %p\n", (void *) &p);
    return 0;
}
```

```
//Saída
*p = 1234
p = 6422300
p = 0061FF1C
&p = 0061FF18
```

Exercícios 03

Ex01: Verifique que a troca de valores de variáveis discutida acima poderia ser obtida por meio de uma macro do pré-processador:

```
//Código
void troca (int *p, int *q)
{
    int temp;
    temp = *p; *p = *q; *q =
}
int *p, *q;
p = &i; q = &j;
troca (p, q);
```

```
//Macro Troca de valores
#define troca (X, Y) {int t = X; X = Y; Y = t;}
. . .
troca (i, j);
```

Resposta:

Sim, é possível realizar a troca de valores usando uma **macro do pré-processador** como a definida. A macro `#define troca(X, Y) {int t = X; X = Y; Y = t;}` substitui X e Y diretamente no código onde é chamada, permitindo trocar os valores das variáveis i e j sem a necessidade de ponteiros.

Problema: A macro `#define troca(X, Y) {int t = X; X = Y; Y = t;}` expande diretamente o código em que X e Y são passados. Isso significa que quando você chama `troca(i, j);`, o pré-processador substitui X e Y por i e j, e o código resultante será: `{int t = i; i = j; j = t;}`

Ex02: Por que o código abaixo está errado?

```
//Código
void troca (int *i, int *j)
{
    int *temp;
    *temp = *i; *i = *j; *j
}
```

Resposta:

Criou-se um ponteiro tempo que não aponta para o endereço de memória de outra variável e ainda recebe um valor. Ponteiros não comportam-se como variáveis.

Uso incorreto de temp:

- `int *temp` declara temp como um ponteiro para int, mas temp não aponta para nenhuma área de memória válida.
- Em seguida, `*temp = *i;` tenta desreferenciar temp para armazenar o valor de *i em um local de memória apontado por temp. Como temp não foi inicializado para apontar para um local de memória válido, isso causa comportamento indefinido. Em muitos casos, isso levará a um erro de segmentação (segmentation fault).

Exercícios 03

Ex01: Suponha que os elementos de um vetor v são do tipo int e cada int ocupa 4 bytes no seu computador. Se o endereço de v[0] é 55000, qual o valor da expressão `v + 3`?

Resposta:

Criou-se um ponteiro tempo que não aponta para o endereço de memória de outra variável e ainda recebe um valor. Ponteiros não comportam-se como variáveis.

Ex02: Suponha que i é uma variável inteira e v um vetor de inteiros. Descreva, em português, a sequência de operações que o computador executa para calcular o valor da expressão `&v[i + 9]`.

Resposta:

Criou-se um ponteiro tempo que não aponta para o endereço de memória de outra variável e ainda recebe um valor. Ponteiros não comportam-se como variáveis.

Para calcular o valor da expressão `&v[i + 9]`, o computador realiza as seguintes operações:

Calcula o valor do índice interno da expressão:

- A expressão `i + 9` é avaliada primeiro.
- O valor de i é somado a 9, resultando em um índice, que chamaremos de n, correspondente ao elemento `v[n]` do vetor.

Determina o deslocamento em memória:

- Cada elemento do vetor v é do tipo `int`. Supondo que cada `int` ocupa 4 bytes, o deslocamento necessário para acessar o elemento $v[n]$ é de $n * 4$ bytes a partir do início do vetor.
- Este deslocamento é então adicionado ao endereço base do vetor v .

Calcula o endereço do elemento $v[i + 9]$:

- O endereço de $v[0]$ (o início do vetor) é somado ao deslocamento calculado acima, resultando no endereço de $v[i + 9]$.

Retorna o endereço calculado:

- A expressão `&v[i + 9]` agora possui o endereço do elemento $v[i + 9]$.

Ex03: Suponha que v é um vetor. Descreva a diferença conceitual entre as expressões $v[3]$ e $v + 3$.

Resposta:

- a. $v[3]$ — Acessa o valor do elemento no índice 3

Conceitualmente, $v[3]$ representa o **conteúdo** do elemento no índice 3 de v , ou seja, o valor armazenado em $v[3]$

- b. $v + 3$ — Calcula o endereço do quarto elemento

Conceitualmente, $v + 3$ representa o **endereço** onde $v[3]$ está localizado, mas não acessa diretamente o valor no índice 3.

Ex04: O que faz a seguinte função?

```
//Código
void imprime (char *v
{
char *c; for (c = v;
printf ("%c", *c);
}
```

Resposta:

A função `imprime` é projetada para imprimir uma sequência de caracteres a partir de um vetor ou string, cujo endereço inicial é passado como argumento. Vamos detalhar o funcionamento da função:

Parâmetros da Função

- `char *v`: Um ponteiro para o início de um vetor de caracteres (string).
- `int n`: O número de caracteres a serem impressos a partir do vetor v .

Funcionamento da Função

Declaração do Ponteiro `c`:

- `char *c`; declara um ponteiro para caracteres (c), que será usado para iterar sobre a sequência de caracteres começando em v .

Loop `for`:

- `for (c = v; c < v + n; c++)` inicializa c com o endereço de v e continua o loop enquanto c for menor que $v + n$.
- Aqui, $v + n$ é o endereço que está n caracteres após o início de v . Portanto, o loop percorre exatamente n caracteres.

Impressão dos Caracteres:

- Dentro do loop, `printf ("%c", *c);` desreferencia o ponteiro c para acessar o caract

Ex05: Analise o seguinte programa → Os valores de x e $v[0]$ não deveriam ser iguais?

```
//Código
void func1 (int x) {
    x = 9 * x;
}

void func2 (int v[]) {
    v[0] = 9 * v[0];
}
```

//O programa produz a seguinte resposta, que é

```
x: 111
v[0]: 999
```

//Os valores de x e $v[0]$ não deveriam ser igua

```
int main (void) {
    int x, v[2];
    x    = 111;
    func1 (x); printf ("x: %d\n", x);
    v[0] = 111;
    func2 (v); printf ("v[0]: %d\n", v[0]);
    return EXIT_SUCCESS;
}
```

Resposta:

Função func1(int x):

- func1 recebe um inteiro x como argumento.
- **Passagem por valor:** Isso significa que x é uma cópia do valor passado. Portanto, qualquer modificação feita a x dentro de func1 não afeta a variável original em main.

Função func2(int v[]):

- func2 recebe um vetor v como argumento.
- **Passagem por referência:** Em C, quando você passa um vetor para uma função, na verdade, você está passando o endereço do primeiro elemento do vetor. Portanto, qualquer modificação feita a v dentro de func2 afeta o vetor original em main.

Resumo da Saída

- **x: 111:** O valor de x não mudou porque func1 não alterou a variável original.
- **v[0]: 999:** O valor de v[0] mudou para 999 porque func2 alterou o conteúdo do vetor original

Ex06: O seguinte fragmento de código pretende decidir se abacate vem lexicograficamente antes ou depois de uva. (Veja a seção Cadeias constantes do capítulo Strings.) O que está errado?

```
//Código
char *a, *b;
a = "abacate"; b = "uva";
if (a < b)
    printf ("%s vem antes de %s\n"
else
    printf ("%s vem depois de %s\n"
```

Resposta:

O problema no código está na comparação if (a < b). Essa comparação não compara as **strings** em si, mas sim os **endereços de memória** para onde a e b apontam. Em C, ao comparar ponteiros com a < b, o compilador verifica se o endereço de a é menor que o de b, e isso não tem nenhuma relação com a ordem lexicográfica das strings.